Chapter Four

Arrays

Problem

- Given 5 numbers, read them in and calculate their average
- THEN print out the ones that were above average

Data Structure Needed

- Need some way to hold onto all the individual data items after processing them
- making individual identifiers x1, x2, x3,... is not practical or flexible
- the answer is to use an ARRAY
- a data structure - bigger than an individual variable or constant

Arrays

- Data structures containing related data items of same type
- Always remain the same size once created

Arrays

- Array
  - Consecutive group of memory locations
    - All of which have the same type
  - Index
    - Position number used to refer to a specific location/element
    - Also called subscript
    - Place in square brackets
      - Must be positive integer or integer expression
    - First element has index zero, last element has index n-1

Properties of an array

- Homogeneous
- Contiguous
- Have random access to any element
- Ordered (numbered from 0 to n-1)
- Number of elements does not change - MUST be a constant when declared

Declaring Arrays

- Declaring an array
  - Arrays occupy space in memory
  - Programmer specifies type and number of elements

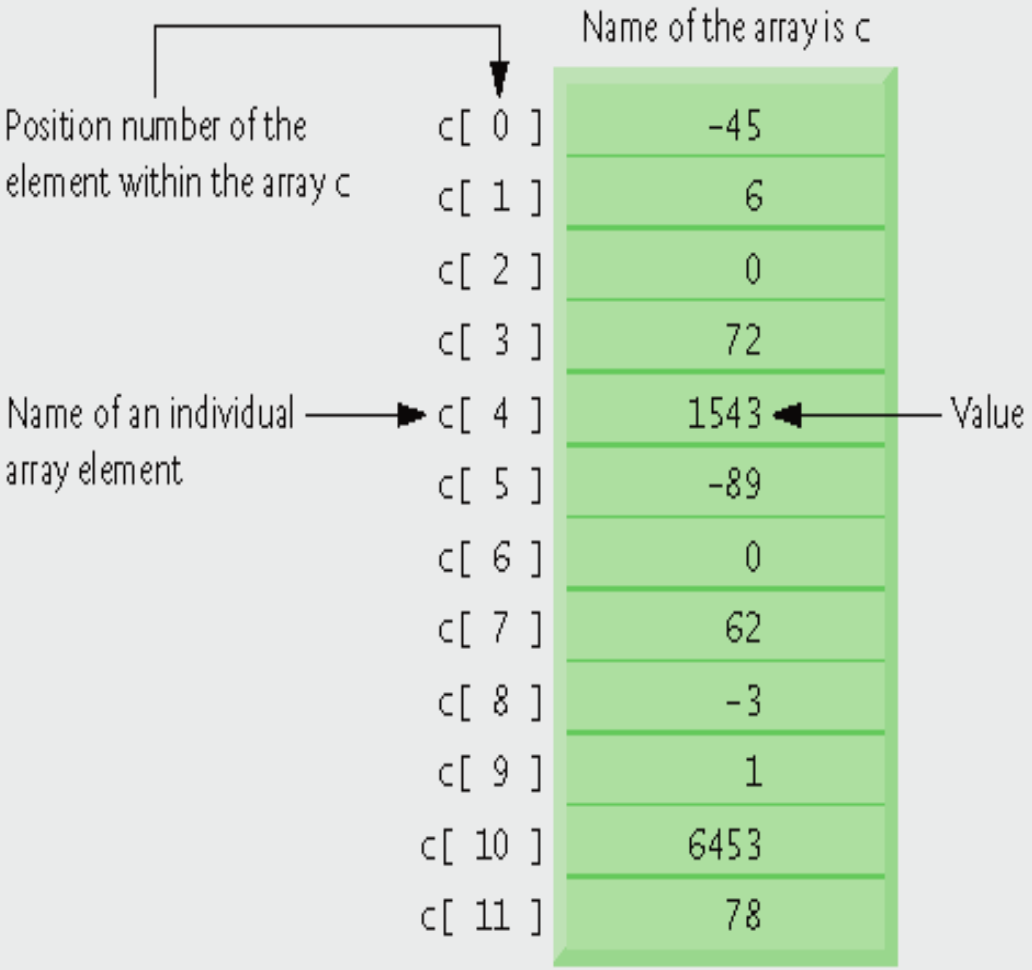
  Syntax

  Datatype array_idef[howmany];

  - Example
    - `int c[ 12 ];`
      - » `c` is an array of 12 `ints`
  - Array's size must be an integer constant greater than zero
  - Multiple arrays of the same type can be declared in a single declaration
    - Use a comma-separated list of names and sizes
    - Example int c[], d[];

Arrays (Cont.)

- To refer to individual elements
  - array_idef [ ] with index in the brackets
    - Example (assume a = 5 and b = 6)
      - c[a + b] += 2;
        - » Adds 2 to array element c[11]


Name of the array is c
Position number of the element within the array c
Name of an individual array element
Value
c[0]
c[1]
c[2]
c[3]
c[4]
c[5]
c[6]
c[7]
c[8]
c[9]
c[10]
c[11]
-45
6
0
72
1543
-89
0
62
-3
1
6453
78

Arrays (Cont.)

- Examine array `c` in the figure
  - `c` is the array *name*
  - `c` has 12 *elements* (`c[0]`, `c[1]`, … `c[11]`)
    - The *value* of `c[0]` is `-45`
- Brackets used to enclose an array subscript are actually an operator in C++

Declaration of an Array

- The index is also called the subscript
- In C++, the first array element always has subscript 0, the second array element has subscript 1, etc.
- The base address of an array is its beginning address in memory

Using a named constant

- it is very common to use a named constant to set the size of an array
- E.g.

```
const int SIZE = 15;
int arr[SIZE];
```

- useful because it can be used to control loops throughout program
- easy to change if size of array needs to be changed

Cont..

•It is important to note the difference between the “seventh element of the array” and “array element 7.” Array subscripts begin at 0, so the “seventh element of the array” has a subscript of 6, while “array element 7” has a subscript of 7 and is actually the eighth element of the array. Unfortunately, this distinction frequently is a source of off-by-one errors. To avoid such errors, we refer to specific array elements explicitly by their array name and subscript number (e.g., `c[ 6 ]` or `c[ 7 ]`).

Solution to problem

```
int counter = 0, n[5], total = 0;
float average;
while (counter < 5)
{
    cout << "enter a number ";
    cin >> n[counter];
    total = total + n[counter];
    counter = counter + 1;
    }
average = total / 5;
    counter = 0;
while (ct < 5)
  {
      if (n[ct] > average)
        cout << n[ct];
      ct = ct + 1;
  }
```

- **Input and Display 5 N**umbers:

```
#include <iostream.h>
int main() {
  int num[5];
 cout << "Enter 5 numbers: ";
 for(int i = 0; i < 5; i++) {
 cin >> num[i];
  }
  cout << "You entered: ";
 for(int i = 0; i < 5; i++) {
 cout << num[i] << " ";
    }
  return 0;
}
```

- //Accept 50 Students' Name, Age, Mark
- #include <iostream.h>

```
int main() {
string name[50];
int age[50];
float mark[50];
cout << "Enter name, age, and mark for 50 students:\n";
  for(int i = 0; i < 50; i++) {
 cout << "\nStudent " << i + 1 << endl;
 cout << "Enter name: ";
cin >> name[i];
cout << "Enter age: ";
 cin >> age[i];
cout << "Enter mark: ";
 cin >> mark[i];
 }}
```

Initializing Array

- Initializing an array in a declaration with an initializer list
  - Initializer list
    - Items enclosed in braces ({})
    - Items in list separated by commas
    - Example
      - `int n[] = { 10, 20, 30, 40, 50 };`
        - » Because array size is omitted in the declaration, the compiler determines the size of the array based on the size of the initializer list
        - » Creates a five-element array
        - » Index values are 0, 1, 2, 3, 4
        - » Initialized to values 10, 20, 30, 40, 50, respectively

Initializing Array

- Initializing an array in a declaration with an initializer list (Cont.)
  - If fewer initializers than elements in the array
    - Remaining elements are initialized to zero
    - Example
      - `int n[ 10 ] = { 0 };`
        - » Explicitly initializes first element to zero
        - » Implicitly initializes remaining nine elements to zero
  - If more initializers than elements in the array
    - Compilation error

Initialization of arrays

- int a[] = {1, 2, 9, 10}; // has 4 elements
- int a[5] = {2, 5, 4, 1, -2, 5}; // error!
- int a[5] = {2, 3}; // rest are zero
- int a[5] = {0}; // all are zero
- can use it with char, float, even bool

Find the Sum of Array Elements

```
#include<iostream>
using namespace std;
Int main()
{
int a[5] = {5, 10, 15, 20, 25};
int sum = 0;
for(int i = 0; i < 5; i++) {
    sum += a[i];
}

cout << "Sum = " << sum;
return 0;
}
```

Find the Sum of Array Elements:

```
#include<iostream>
using namespace std;
Int main()
{
int a[5] = {12, 45, 2, 67, 23};
int max = a[0];
for(int i = 1; i < 5; i++) {
 if(a[i] > max)
  max = a[i];
}
cout << "Maximum = " << max;
```

Watch out index out of range!

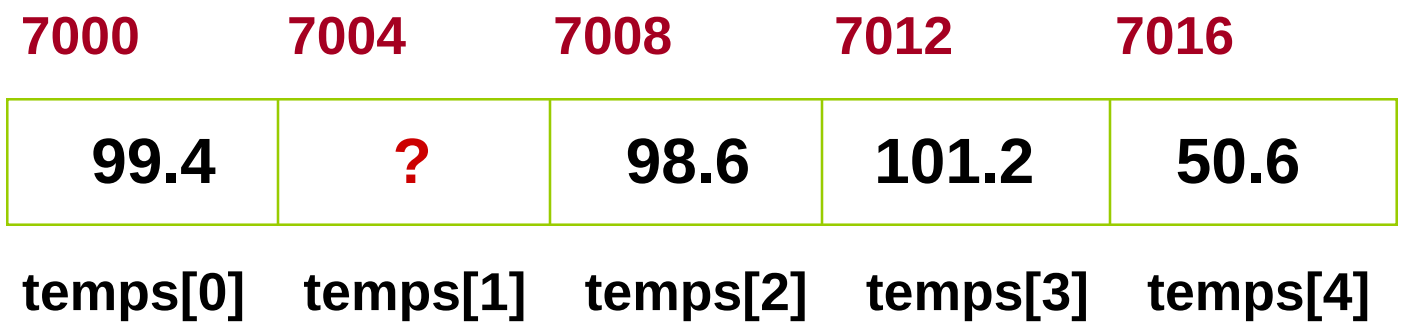
- subscripts range from 0 to n-1
- the compiler will NOT tell you if an index goes out of that range - it cannot detect
- can make very bad bugs
  - from just garbage values in your data to
  - crashing your computer to
  - damaging your hard drive!

```
// Initializing an array.
#include <iostream.h>
#include <iomanip.h>
   int main()
   {
      int n[ 10 ]; // n is an array of 10 integers

      // initialize elements of array n to 0
      for ( int i = 0; i < 10; i++ )
         n[ i ] = 0; // set element at location i to 0

   // output each array element's value
      for ( int j = 0; j < 10; j++ )
         cout << setw( 7 ) << j << setw( 13 ) << n[ j ] << endl;

      return 0; // indicates successful termination
   } // end main
```

```
 Element         Value
       0             0
       1             0
       2             0
       3             0
       4             0
       5             0
       6             0
       7             0
       8             0
       9             0
```

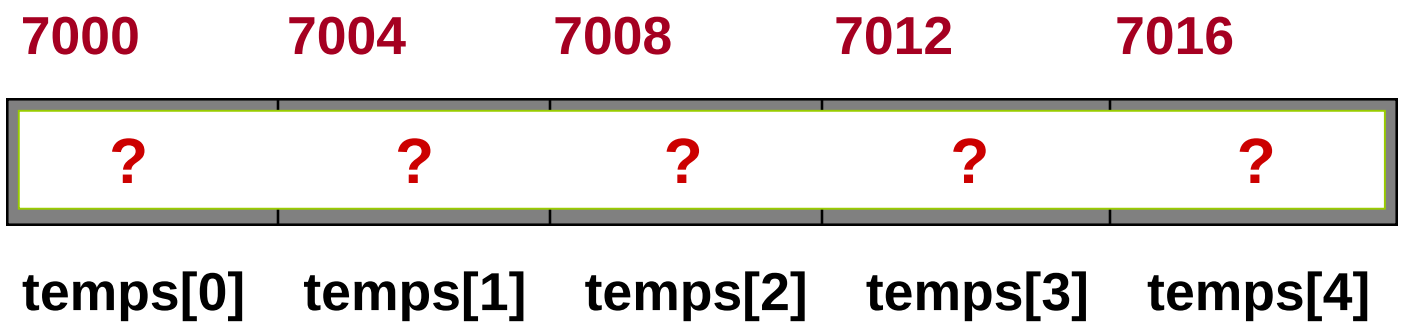
Assigning Values to Individual Array Elements

```
float temps[5];
int m = 4; // Allocates memory
temps[2] = 98.6;
temps[3] = 101.2;
temps[0] = 99.4;
temps[m] = temps[3] / 2.0;
temps[1] = temps[3] - 1.2;
// What value is assigned?
```



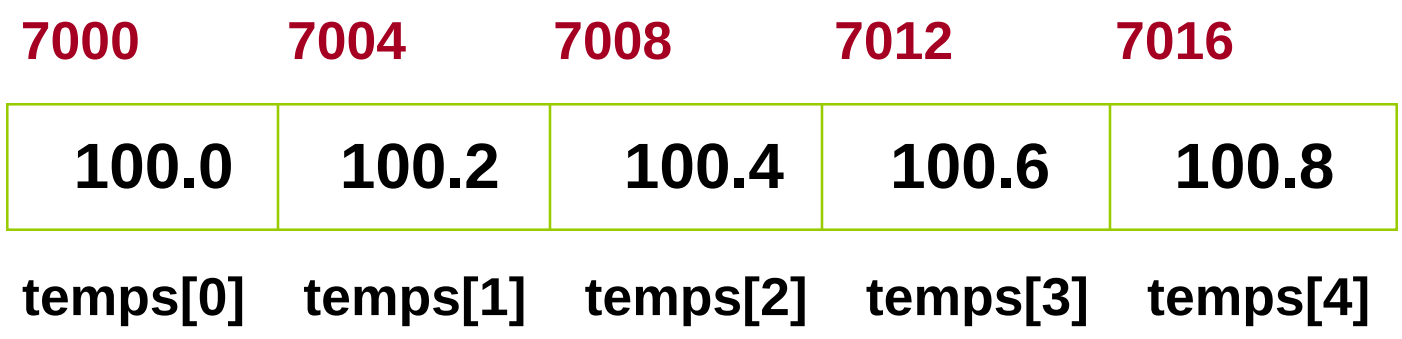
What values are assigned?

```
float temps[5]; // Allocates memory
int m;

for (m = 0; m < 5; m++)
{
    temps[m] = 100.0 + m * 0.2 ;
}
```



Now what values are printed?

```
float temps[5];          // Allocates memory
int m;
 . . . . .
for (m = 4; m >= 0; m--)     //display array in reverse order
{
     cout  <<  temps[m] << endl;
}
```



Function to print all elements of an array

- #include<iostream>

```
using namespace std;
void printArray(int A[], int n);
int main() {
 int nums[5] = {10, 20, 30, 40, 50};
 printArray(nums, 5);
}
void printArray(int arr[], int size)
{
   for (int i = 0; i < size; i++)
    cout << arr[i] << " ";

}
```

Function to find the Largest element in an array

```
#include<iostream.h>
int findMax(int arr[], int size);
int main() {
 int values[5] = {12, 45, 7, 89, 30};
  cout << findMax(values, 5);
}
int findMax(int arr[], int size) {
 int max = arr[0];
 for (int i = 1; i < size; i++) {
  if (arr[i] > max)
  max = arr[i];
 }
    return max;
}
```

Function to find the sum of array elements

- #include<iostream.h>

```
int sumArray(int arr[], int size)
{
  int sum = 0;
  for (int i = 0; i < size; i++) {
      sum += arr[i];
    }
    return sum;
}
int main() {
    int a[4] = {5, 10, 15, 20};
    cout << sumArray(a, 4);
}
```

Indexes

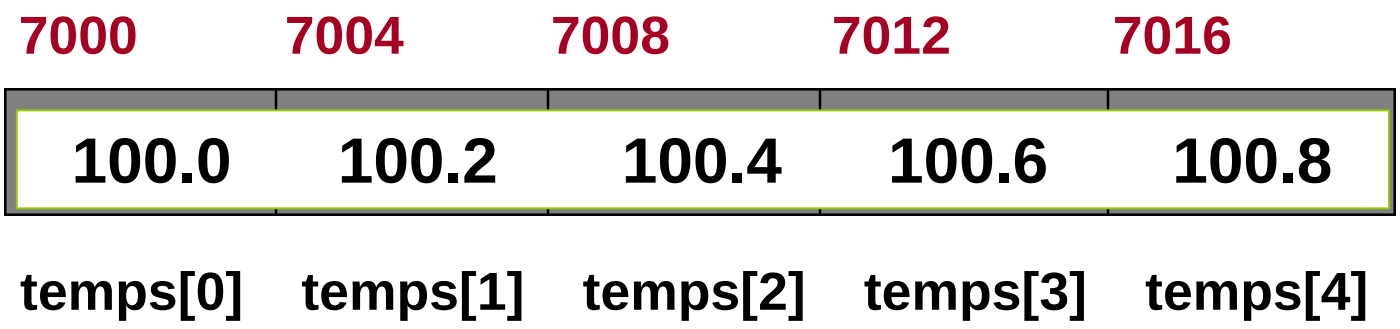
- subscripts can be constants or variables or expressions
- if i is 5, a[i-1] refers to a[4] and a[i*2] refers to a[10]
- you can use **i** as a subscript at one point in the program and **j** as a subscript for the same array later - only the value of the variable matters

Variable Subscripts

```
float temps[5];          // Allocates memory
int m = 3;
  .   .   .   .   .   .
```

What is temps[m + 1] ?

What is temps[m] + 1 ?



```cpp
// Fig. 7.5: fig07_05.cpp
// Set array s to the even integers from 2 to 20.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

int main()
{
   // constant variable can be used to specify array size
   const int arraySize = 10;

   int s[ arraySize ]; // array s has 10 elements

   for ( int i = 0; i < arraySize; i++ ) // set the values
      s[ i ] = 2 + 2 * i;
```

Random access of elements

- problem : read in numbers from a file, only single digits - and count them - report how many of each there were
- use an array as a set of counters
  - ctr [0] is how many zero's, ctr[1] is how many ones, etc.
- **ctr[num] ++;** is the crucial statement

Selection sort - 1-d array

Algorithm for the sort

1. find the maximum in the list
2. put it in the highest numbered element by swapping it with the data that was at that location
3. repeat 1 and 2 for shorter unsorted list - not including highest numbered location
4. repeat 1-3 until list goes down to one

Find the maximum in the list

```
// n is number of elements
max = a[0];    // value of largest element
                          // seen so far
for (i = 1; i < n; i++) // note start at 1, not 0
   if (max < a[i])
        max = a[i];
// now max is value of largest element in list
```

Find the location of the max

```
max = 0;   // max is now location of the max
for (i = 1; i < n; i++)
   if (a[max] < a[i])
        max = i;
```

Swap with highest numbered

element at right end of list is numbered n-1

```
temp = a[max];
a[max] = a[n-1];
a[n-1] = temp;
```

Find next largest element and swap

```
max = 0;
for (i = 1; i < n-1; i++)  // note n-1, not n
   if (a[max] < a[i])
                max = i;
temp = a[max];
a[max] = a[n-2];
a[n-2] = temp;
```

put a loop around the general code to repeat for n-1 passes

```
for (pass = n-1; pass >= 0; pass --)
{  max = 0;
   for (i = 1; i < pass; i++)
        if (a[max] < a[i])
              max = i;
   temp = a[max];
   a[max] = a[pass];
   a[pass] = temp;
}
```

```
  // Linear search of an array.
#include <iostream.h>
  int main()
  {
     const int arraySize = 100; // size of array a
     int a[ arraySize ]; // create array a
     int searchKey; // value to locate in array a

     for ( int i = 0; i < arraySize; i++ )
        a[ i ] = 2 * i; // create some data

     cout << "Enter integer search key: ";
     cin >> searchKey;
      // attempt to locate searchKey in array a

     for ( int j = 0; j < arraySize; j++ )
        if ( array[ j ] == searchkey ) // if found,
           return j; // return location of key

     return -1; // key not found
```

Function to count how many even numbers are in an arra

```
#include<iostream.h>
int countEven(int arr[], int size) {
  int count = 0;
for (int i = 0; i < size; i++) {
  if (arr[i] % 2 == 0) {
 count++;
   }
  }
   return count;
}
int main() {
    int arr[6] = {1, 2, 3, 4, 6, 9};
    cout << countEven(arr, 6);}
```

Multidimensional Array

- Multidimensional arrays with two dimensions
  - Called two dimensional or 2-D arrays
  - Represent tables of values with rows and columns
  - Elements referenced with two subscripts (`[x][y]`)
  - In general, an array with *m* rows and *n* columns is called an *m*-by-*n* array
  - E.g.
    - int a[5][4]; // row then column
    - twenty elements, numbered from [0][0] to [4][3]
  - Multidimensional arrays can have more than two dimensions

Two-Dimensional Array

- **A two-dimensional array is a collection of components, all of the same type, structured in two dimensions, (referred to as rows and columns)**
- **Individual components are accessed by a pair of indexes representing the component's position in each dimension**

DataType ArrayName[ConstIntExpr][ConstIntExpr]...;

Processing a 2-d array by rows

finding the total for the first row

```
    for (i = 0; i < 5; i++)
        total = total + a[0][i];
```

finding the total for the second row

```
    for (i = 0; i < 5; i++)
        total = total + a[1][i];
```

Processing a 2-d array by rows

total for ALL elements by adding first row, then second row, etc.

```
for (i = 0; i < 5; i++)
    for (j = 0; j < 4; j++)
        total = total + a[i][j];
```

Processing a 2-d array by columns

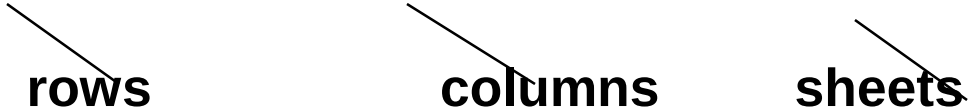
total for ALL elements by adding first column, second column, etc.

```
for (j = 0; j < 4; j++)
    for (i = 0; i < 5; i++)
        total = total + a[i][j];
```

Declaring Multidimensional Arrays

Example of three-dimensional array

```
const NUM_DEPTS = 5;
// mens, womens, childrens, electronics, furniture
const NUM_MONTHS = 12;
const NUM_STORES = 3; // White Marsh,  Owings Mills, Towson

int  monthlySales[NUM_DEPTS][NUM_MONTHS][NUM_STORES];
```



Declare and Initialize

- #include <iostream>

```
using namespace std;
int main() {
// 2D array with 2 rows and 3 columns
  int arr[2][3] = { {1, 2, 3}, {4, 5, 6} };
Or int arr[2][3] = { 1, 2, 3, 4, 5, 6 };

for (int i = 0; i < 2; i++) {          // loop over rows
  for (int j = 0; j < 3; j++) {       // loop over columns
     cout << arr[i][j] << " ";
    }
  cout << endl;   }
 return 0;
}
```

Accessing Elements:
cout << arr[0][1]; // prints 2 (row 0, column 1)
arr[1][2] = 10; //// sets element in row 1, col 2 to 10